

Database Management Systems Lab

[ENCS254]

Lab File Submitted to



Bachelor of Technology

In

Computer Science Engineering

SEMESTER – 4th

Submitted by

Ananya Sharma – 2401010056

SECTION - B

Course Teacher

Dr. Swati Gupta

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

Table of Contents

S.No	Lab Practicals	Page No.
1	To Perform basic SQL Commands create, insert, select, where Clause	1-5
2	To understand basic operators like and or , not	6-10
3	To perform basic SQL command Order by	11-14
4	To perform basic SQL Command update alter delete drop	15-19
5	To perform all joins	20-25
6	To perform Set Operator	26-30
7	To perform SQL Like Operator	31-36
8	To perform SQL Constraints	37-39
9.	To perform aggregation function in SQL	40-44

10.	To perform Subquery	45-48
11.	To perform String Functions	49-57

Experiment No – 1

Aim- To perform basic SQL operations using commands like CREATE, INSERT INTO, SELECT, and WHERE clause.

Theory

SQL (Structured Query Language) is used to manage and manipulate databases.

- **CREATE → Used to create a table**
- **INSERT INTO → Used to insert data into a table**
- **SELECT → Used to retrieve data**
- **WHERE → Used to filter records based on condition**

Procedure

- 1. Open SQL environment (MySQL / SQLite / Oracle etc.)**
- 2. Create a database (optional)**
- 3. Create a table**
- 4. Insert records**
- 5. Retrieve data using SELECT**
- 6. Apply WHERE condition**

SQL Commands

Create Table

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Student (  
2     ID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Age INT,  
5     Course VARCHAR(50)  
6 );
```

Insert Values

```
INSERT INTO Student (ID, Name, Age, Course) VALUES  
(1, 'Ananya', 20, 'BCA'),  
(2, 'Rahul', 21, 'BBA'),  
(3, 'Priya', 19, 'BCA'),  
(4, 'Amit', 22, 'BTech');
```

Display All Records

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Student (  
2     ID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Age INT,  
5     Course VARCHAR(50)  
6 );  
7 INSERT INTO Student (ID, Name, Age, Course) VALUES  
8 (1, 'Ananya', 20, 'BCA'),  
9 (2, 'Rahul', 21, 'BBA'),  
10 (3, 'Priya', 19, 'BCA'),  
11 (4, 'Amit', 22, 'BTech');  
12 SELECT * FROM Student;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download

Execution time: 0.001 seconds

	ID	NAME	AGE	COURSE
1	1	Ananya	20	BCA
2	2	Rahul	21	BBA
3	3	Priya	19	BCA
4	4	Amit	22	BTech

Use Where Clause

Ex 1 - Choose Specific Course

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Student (  
2     ID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Age INT,  
5     Course VARCHAR(50)  
6 );  
7 INSERT INTO Student (ID, Name, Age, Course) VALUES  
8 (1, 'Ananya', 20, 'BCA'),  
9 (2, 'Rahul', 21, 'BBA'),  
10 (3, 'Priya', 19, 'BCA'),  
11 (4, 'Amit', 22, 'BTech');  
12 SELECT * FROM Student WHERE Course = 'BCA';
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.117 seconds

	ID	NAME	AGE	COURSE
1	1	Ananya	20	BCA
2	3	Priya	19	BCA

Ex 2 – Age greater than 20

```
12 SELECT Name, Age FROM Student WHERE Age > 20;
```

	NAME	AGE
1	Rahul	21
2	Amit	22

Conclusion - Basic SQL commands were successfully executed. We learned how to create a table, insert data, and retrieve specific records using SELECT and WHERE clause.

Experiment No – 2

Aim- To study and implement the WHERE clause combined with AND, OR, and NOT operators in SQL queries.

Theory

The WHERE clause is used to filter records from a table based on conditions.

It can be combined with logical operators:

- **AND** → Returns records when *all conditions are true*
- **OR** → Returns records when *any one condition is true*
- **NOT** → Returns records when *condition is false*

Procedure

1. Open SQL environment
2. Create a table
3. Insert sample data
4. Apply WHERE clause with AND, OR, NOT operators
5. Execute and observe results

SQL Commands

1. Create Table

[SQL Worksheet]* ▼



Aa ▼



```
1 CREATE TABLE Employee (  
2     EmpID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Department VARCHAR(50),  
5     Salary INT  
6 );
```

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SQL> CREATE TABLE Employee (  
      EmpID INT PRIMARY KEY,  
      Name VARCHAR(50),  
      Department VARCHAR(50),...  
Show more...
```

Table EMPLOYEE created.

2 Insert Data

```
7 INSERT INTO Employee VALUES  
8 (101, 'Rohit', 'HR', 30000),  
9 (102, 'Neha', 'IT', 50000),  
10 (103, 'Arjun', 'Finance', 45000),  
11 (104, 'Meena', 'IT', 60000),  
12 (105, 'Karan', 'HR', 35000);
```


3 Display All Records

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Employee (  
2     EmpID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Department VARCHAR(50),  
5     Salary INT  
6 );  
7 INSERT INTO Employee VALUES  
8 (101, 'Rohit', 'HR', 30000),  
9 (102, 'Neha', 'IT', 50000),  
10 (103, 'Arjun', 'Finance', 45000),  
11 (104, 'Meena', 'IT', 60000),  
12 (105, 'Karan', 'HR', 35000);  
13 SELECT * FROM Employee
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.126 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	101	Rohit	HR	30000
2	102	Neha	IT	50000
3	103	Arjun	Finance	45000
4	104	Meena	IT	60000
5	105	Karan	HR	35000

4 Queries using WHERE with Operators

1. WHERE with AND

IT department and salary > 50000

```

13 SELECT * FROM Employee
14 WHERE Department = 'IT' AND Salary > 50000;

```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.007 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	104	Meena	IT	60000

2. WHERE with OR

HR department or salary > 55000

```

13 SELECT * FROM Employee
14 WHERE Department = 'HR' OR Salary > 55000;

```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.006 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	101	Rohit	HR	30000
2	104	Meena	IT	60000
3	105	Karan	HR	35000

3. WHERE with NOT

Except IT department Display All Records

```
13 SELECT * FROM Employee
14 WHERE NOT Department = 'IT';
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.121 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	101	Rohit	HR	30000
2	103	Arjun	Finance	45000
3	105	Karan	HR	35000

4. Combined AND + NOT

```
13 SELECT * FROM Employee
14 WHERE Department = 'HR' AND NOT Salary < 35000;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.006 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	105	Karan	HR	35000

5. Combined OR + NOT

```
13 SELECT * FROM Employee
14 WHERE NOT Department = 'Finance' OR Salary < 40000;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download

Execution time: 0.009 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	101	Rohit	HR	30000
2	102	Neha	IT	50000
3	104	Meena	IT	60000
4	105	Karan	HR	35000

Conclusion

The WHERE clause was successfully used with AND, OR, and NOT operators to filter records based on multiple conditions.

Experiment No – 3

Aim- To study and implement the ORDER BY clause in SQL to sort records in ascending and descending order.

Theory

The ORDER BY clause is used to sort the result set of a query based on one or more columns.

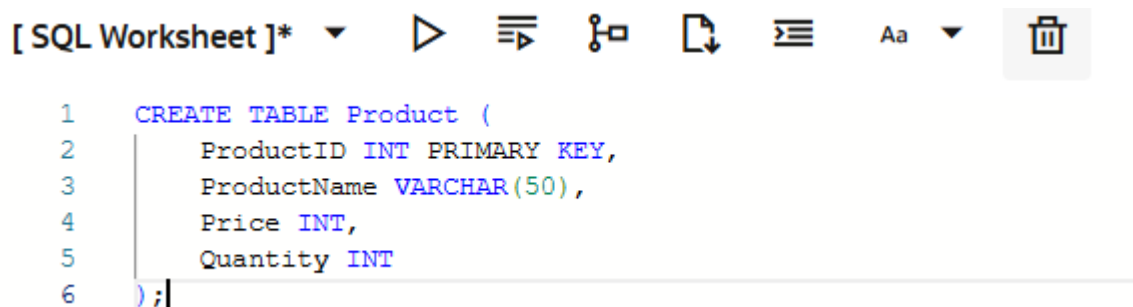
- ASC (Ascending) → Default order (small to large / A to Z)
- DESC (Descending) → Reverse order (large to small / Z to A)

Procedure

1. Open SQL environment
2. Create a table
3. Insert sample data
4. Use SELECT statement with ORDER BY clause
5. Execute and observe sorted results

SQL Commands

1 Create Table



The screenshot shows an SQL Worksheet interface. At the top, there is a toolbar with icons for running queries, saving, and other functions. Below the toolbar, the SQL command is entered in a text area:

```
1 CREATE TABLE Product (  
2     ProductID INT PRIMARY KEY,  
3     ProductName VARCHAR(50),  
4     Price INT,  
5     Quantity INT  
6 );
```

2 Insert Data

```
7  INSERT INTO Product VALUES
8  (1, 'Laptop', 60000, 10),
9  (2, 'Mouse', 500, 50),
10 (3, 'Keyboard', 1500, 30),
11 (4, 'Monitor', 12000, 15),
12 (5, 'Printer', 8000, 8);
```

3 Display All Records

[SQL Worksheet]*      Aa 

```
1  CREATE TABLE Product (
2      ProductID INT PRIMARY KEY,
3      ProductName VARCHAR(50),
4      Price INT,
5      Quantity INT
6  );
7  INSERT INTO Product VALUES
8  (1, 'Laptop', 60000, 10),
9  (2, 'Mouse', 500, 50),
10 (3, 'Keyboard', 1500, 30),
11 (4, 'Monitor', 12000, 15),
12 (5, 'Printer', 8000, 8);
13 SELECT * FROM Product
```

Query result					Script output	DBMS output	Explain Plan	SQL history
		Download	▼	Execution time: 0.008 seconds				
	PRODUCTID	PRODUCTNAME	PRICE	QUANTITY				
1	1	Laptop	60000	10				
2	2	Mouse	500	50				
3	3	Keyboard	1500	30				
4	4	Monitor	12000	15				
5	5	Printer	8000	8				

Queries using ORDER BY

1. Sort by Price (Ascending Order)

```
13 SELECT * FROM Product
14 ORDER BY Price ASC;
```

	PRODUCTID	PRODUCTNAME	PRICE	QUANTITY
1	2	Mouse	500	50
2	3	Keyboard	1500	30
3	5	Printer	8000	8
4	4	Monitor	12000	15
5	1	Laptop	60000	10

2. Sort by Price (Descending Order)

```
13 SELECT * FROM Product
14 ORDER BY Price DESC;
```

	PRODUCTID	PRODUCTNAME	PRICE	QUANTITY
1	1	Laptop	60000	10
2	4	Monitor	12000	15
3	5	Printer	8000	8
4	3	Keyboard	1500	30
5	2	Mouse	500	50

3. Sort by Product Name (Alphabetically)

```
14 ORDER BY ProductName ASC;
```

	PRODUCTID	PRODUCTNAME	PRICE	QUANTITY
1	3	Keyboard	1500	30
2	1	Laptop	60000	10
3	4	Monitor	12000	15
4	2	Mouse	500	50
5	5	Printer	8000	8

4. Sort by Multiple Columns

```
14 ORDER BY Price ASC, Quantity DESC;
```

	PRODUCTID	PRODUCTNAME	PRICE	QUANTITY
1	2	Mouse	500	50
2	3	Keyboard	1500	30
3	5	Printer	8000	8
4	4	Monitor	12000	15
5	1	Laptop	60000	10

Conclusion

The **ORDER BY** clause was successfully used to sort records in ascending and descending order based on different columns.

Experiment No – 4

Aim- To study and implement basic SQL commands: UPDATE, ALTER, DELETE, and DROP.

Theory

SQL provides different commands to modify database structure and data:

- **UPDATE → Used to modify existing records in a table**
- **ALTER → Used to modify table structure (add, modify, delete columns)**
- **DELETE → Used to remove records from a table**
- **DROP → Used to delete the entire table**

Procedure

- 1. Open SQL environment**
- 2. Create a table**
- 3. Insert sample data**
- 4. Apply UPDATE, ALTER, DELETE, and DROP commands**
- 5. Execute and observe results**

SQL Commands

1 Create table

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Library (  
2     BookID INT PRIMARY KEY,  
3     Title VARCHAR(50),  
4     Author VARCHAR(50),  
5     Price INT  
6 );
```

2 Insert Data

```
7 INSERT INTO Library VALUES  
8 (1, 'Database System', 'Navathe', 500),  
9 (2, 'Operating System', 'Galvin', 650),  
10 (3, 'Computer Networks', 'Tanenbaum', 700),  
11 (4, 'Data Structures', 'Sahni', 400);
```

3 Display All Records

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Library (  
2     BookID INT PRIMARY KEY,  
3     Title VARCHAR(50),  
4     Author VARCHAR(50),  
5     Price INT  
6 );  
7 INSERT INTO Library VALUES  
8 (1, 'Database System', 'Navathe', 500),  
9 (2, 'Operating System', 'Galvin', 650),  
10 (3, 'Computer Networks', 'Tanenbaum', 700),  
11 (4, 'Data Structures', 'Sahni', 400);  
12 SELECT * FROM LIBRARY
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download

Execution time: 0.01 seconds

	BOOKID	TITLE	AUTHOR	PRICE
1	1	Database System	Navathe	500
2	2	Operating System	Galvin	650
3	3	Computer Networks	Tanenbaum	700
4	4	Data Structures	Sahni	400

1 Update Command

Update price of BookID = 2

```
12 UPDATE Library
13 SET Price = 700
14 WHERE BookID = 2;
15 SELECT * FROM LIBRARY
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.005 seconds

	BOOKID	TITLE	AUTHOR	PRICE
1	1	Database System	Navathe	500
2	2	Operating System	Galvin	700
3	3	Computer Networks	Tanenbaum	700
4	4	Data Structures	Sahni	400

2. ALTER Command

Add new column

```
12 ALTER TABLE Library
13 ADD Publisher VARCHAR(50);
14 SELECT * FROM LIBRARY
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.007 seconds

	BOOKID	TITLE	AUTHOR	PRICE	PUBLISHER
1	1	Database System	Navathe	500	(null)
2	2	Operating System	Galvin	700	(null)
3	3	Computer Networks	Tanenbaum	700	(null)
4	4	Data Structures	Sahni	400	(null)

Modify column

```
12 ALTER TABLE Library
13 MODIFY Price FLOAT;
14 SELECT * FROM LIBRARY
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.005 seconds

	BOOKID	TITLE	AUTHOR	PRICE	PUBLISHER
1	1	Database System	Navathe	500	(null)
2	2	Operating System	Galvin	700	(null)
3	3	Computer Networks	Tanenbaum	700	(null)
4	4	Data Structures	Sahni	400	(null)

Drop column

```
12 ALTER TABLE Library
13 DROP COLUMN Publisher;
14 SELECT * FROM LIBRARY
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.117 seconds

	BOOKID	TITLE	AUTHOR	PRICE
1	1	Database System	Navathe	500
2	2	Operating System	Galvin	700
3	3	Computer Networks	Tanenbaum	700
4	4	Data Structures	Sahni	400

3. DELETE Command

Delete a record

```
12 DELETE FROM Library
13 WHERE BookID = 1;
14 SELECT * FROM LIBRARY
```

Query result Script output DBMS output Explain Plan SQL history				
Download Execution time: 0.001 seconds				
	BOOKID	TITLE	AUTHOR	PRICE
1	2	Operating System	Galvin	700
2	3	Computer Networks	Tanenbaum	700
3	4	Data Structures	Sahni	400

4. DROP Command

Delete entire table

```
12 DROP TABLE Library;
```

```
SQL> DROP TABLE Library
```

```
Table LIBRARY dropped.
```

Conclusion

SQL commands UPDATE, ALTER, DELETE, and DROP were successfully performed on the Library table.

Experiment No – 5

Aim- To study and implement different types of SQL JOIN operations to combine data from multiple tables.

Theory

SQL JOINS are used to retrieve data from two or more tables based on a related column.

Types of JOINS:

- **INNER JOIN** → Returns matching records from both tables
- **LEFT JOIN** → Returns all records from left table + matched from right
- **RIGHT JOIN** → Returns all records from right table + matched from left
- **FULL JOIN** → Returns all records from both tables (matched + unmatched)

Procedure

1. **Open SQL environment**
2. **Create two related tables**
3. **Insert data**
4. **Apply different JOIN queries**
5. **Observe results**

SQL Commands

1 Create Table

Table 1: Employee

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE employees (  
2     employee_id NUMBER PRIMARY KEY,  
3     name VARCHAR2(100),  
4     department_id NUMBER  
5 );
```

Table 2: Department

```
6 CREATE TABLE departments (  
7     department_id NUMBER PRIMARY KEY,  
8     department_name VARCHAR2(100)  
9 );
```

2 Insert Data

Employee Table

```
INSERT ALL  
  INTO employees VALUES (1, 'John Doe', 1)  
  INTO employees VALUES (2, 'Jane Smith', 2)  
  INTO employees VALUES (3, 'Alice Johnson', 1)  
  INTO employees VALUES (4, 'Bob Brown', 3)
```

Department Table

```
16 INSERT ALL  
17     INTO departments VALUES (1, 'HR')  
18     INTO departments VALUES (2, 'Marketing')  
19     INTO departments VALUES (3, 'Finance')  
20 SELECT * FROM dual;  
21  
22 COMMIT;
```

SQL Join Queries

1 Inner Join

Only matching records

```
22 COMMIT;  
23 SELECT e.name, d.department_name  
24 FROM employees e  
25 INNER JOIN departments d  
26 ON e.department_id = d.department_id;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.009 seconds

	NAME	DEPARTMENT_NAME
1	John Doe	HR
2	Alice Johnson	HR
3	Jane Smith	Marketing
4	Bob Brown	Finance

2. LEFT JOIN



All employees + matching department


```
23 SELECT e.name, d.department_name
24 FROM employees e
25 LEFT JOIN departments d
26 ON e.department_id = d.department_id;
```

Query result Script output DBMS output Explain Plan SQL history



Download



Execution time: 0.005 seconds

	NAME	DEPARTMENT_NAME
1	John Doe	HR
2	Alice Johnson	HR
3	Jane Smith	Marketing
4	Bob Brown	Finance

3. RIGHT JOIN



All departments + matching employees

```
23 SELECT e.name, d.department_name
24 FROM employees e
25 RIGHT JOIN departments d
26 ON e.department_id = d.department_id;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.005 seconds

	NAME	DEPARTMENT_NAME
1	John Doe	HR
2	Alice Johnson	HR
3	Jane Smith	Marketing
4	Bob Brown	Finance

4. FULL JOIN

All records from both tables

```

23 SELECT e.name, d.department_name
24 FROM employees e
25 FULL OUTER JOIN departments d
26 ON e.department_id = d.department_id;

```

Query result

Script output

DBMS output

Explain Plan



Download



Execution time: 0.004 seconds

	NAME	DEPARTMENT_NAME
1	John Doe	HR
2	Jane Smith	Marketing
3	Alice Johnson	HR
4	Bob Brown	Finance

Conclusion- Different types of JOIN operations were successfully performed to retrieve data from multiple tables. "All employees are assigned to valid departments and no unmatched records exist."

Experiment No – 6

Aim- To study and implement SQL Set Operators such as UNION, UNION ALL, INTERSECT, and MINUS.

Theory

Set operators are used to combine results from two or more SELECT queries.

- **UNION** → Combines results and removes duplicates
- **UNION ALL** → Combines results and keeps duplicates
- **INTERSECT** → Returns common records
- **MINUS** → Returns records from first query not in second

👉 **Condition:** Number of columns and data types must be same in both queries.

Procedure

1. Open SQL environment
2. Create tables
3. Insert data
4. Apply set operators
5. Observe results

Table Creation

Table 1: Customer_India

[SQL Worksheet]*      Aa  

```
1 CREATE TABLE Customer_India (  
2     ID INT,  
3     Name VARCHAR(50)  
4 );
```

Table 2: Customer_USA

```
5 CREATE TABLE Customer_USA (  
6     ID INT,  
7     Name VARCHAR(50)  
8 );
```

Insert Data

Customer_India

```
9  
9 INSERT INTO Customer_India VALUES  
10 (1, 'Riya'),  
11 (2, 'Arjun'),  
12 (3, 'Kabir');
```

Customer_USA

```
13 INSERT INTO Customer_USA VALUES  
14 (3, 'Kabir'),  
15 (4, 'John'),  
16 (5, 'Emma');
```

Set Operator Queries

1. UNION

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Customer_India (  
2     ID INT,  
3     Name VARCHAR(50)  
4 );  
5 CREATE TABLE Customer_USA (  
6     ID INT,  
7     Name VARCHAR(50)  
8 );  
9 INSERT INTO Customer_India VALUES  
10 (1, 'Riya'),  
11 (2, 'Arjun'),  
12 (3, 'Kabir');  
13 INSERT INTO Customer_USA VALUES  
14 (3, 'Kabir'),  
15 (4, 'John'),  
16 (5, 'Emma');  
17 SELECT * FROM Customer_India  
18 UNION  
19 SELECT * FROM Customer_USA;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

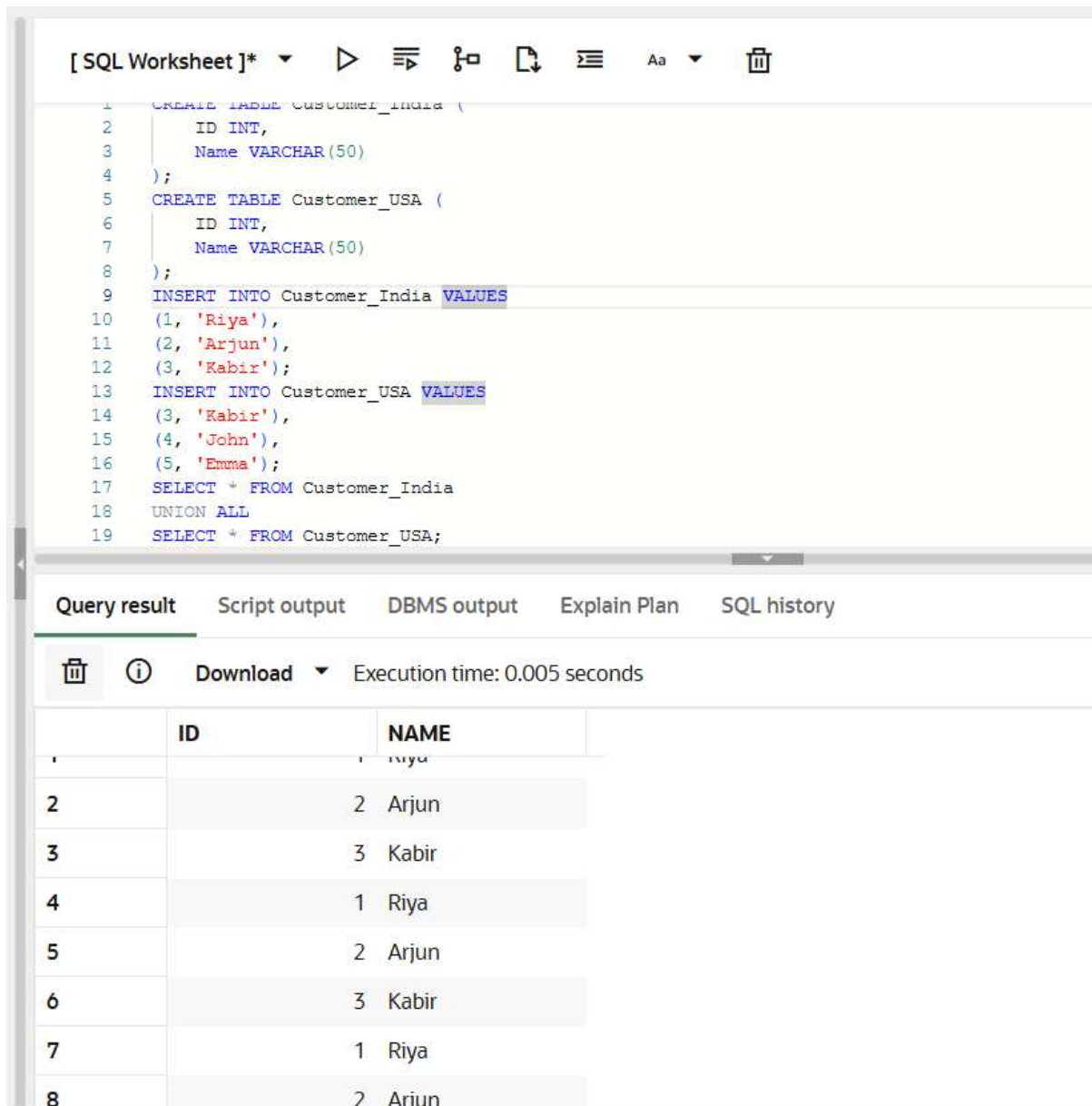


Download

Execution time: 0.123 seconds

	ID	NAME
1	1	Riya
2	2	Arjun
3	3	Kabir
4	4	John
5	5	Emma

2. UNION ALL



The screenshot shows an SQL IDE interface. The top toolbar includes icons for running, saving, and other standard SQL tool functions. The main editor contains the following SQL script:

```
1 CREATE TABLE Customer_India (  
2     ID INT,  
3     Name VARCHAR(50)  
4 );  
5 CREATE TABLE Customer_USA (  
6     ID INT,  
7     Name VARCHAR(50)  
8 );  
9 INSERT INTO Customer_India VALUES  
10 (1, 'Riya'),  
11 (2, 'Arjun'),  
12 (3, 'Kabir');  
13 INSERT INTO Customer_USA VALUES  
14 (3, 'Kabir'),  
15 (4, 'John'),  
16 (5, 'Emma');  
17 SELECT * FROM Customer_India  
18 UNION ALL  
19 SELECT * FROM Customer_USA;
```

Below the editor, the 'Query result' tab is active. It displays the execution time as 0.005 seconds. The result is shown in a table with two columns: ID and NAME. The table contains 8 rows, which are the combined results of the two tables, including duplicates.

ID	NAME
1	Riya
2	Arjun
3	Kabir
4	Riya
5	Arjun
6	Kabir
7	Riya
8	Arjun

3. INTERSECT

```
17 SELECT * FROM Customer_India
18 INTERSECT
19 SELECT * FROM Customer_USA;
```

Query result

Script output

DBMS output

Explain Plan



Download



Execution time: 0.008 seconds

	ID	NAME
1	3	Kabir

4. MINUS

```
17 SELECT * FROM Customer_India
18 MINUS
19 SELECT * FROM Customer_USA;
```

Query result

Script output

DBMS output

Explain Plan



Download



Execution time: 0.005 seconds

	ID	NAME
1	1	Riya
2	2	Arjun

Conclusion- Set operators were successfully used to combine and filter data from multiple tables.

Experiment No – 7

Aim- To study and implement the LIKE operator in SQL for pattern matching.

Theory

The LIKE operator is used in SQL to search for a specified pattern in a column.

Wildcards used with LIKE:

- % (percent) → Represents zero, one, or multiple characters
- _ (underscore) → Represents exactly one character

Procedure

1. Open SQL environment
2. Create a table
3. Insert sample data
4. Apply LIKE operator with different patterns
5. Observe results

Table Creation

[SQL Worksheet]*      Aa  

```
1 CREATE TABLE Employee_Details (  
2     EmpID INT,  
3     Name VARCHAR(50),  
4     City VARCHAR(50)  
5 );
```

Insert Data

```
6  INSERT INTO Employee_Details VALUES
7  (1, 'Ananya', 'Delhi'),
8  (2, 'Rahul', 'Mumbai'),
9  (3, 'Amit', 'Delhi'),
10 (4, 'Neha', 'Chennai'),
11 (5, 'Arjun', 'Delhi');
```

Queries using LIKE Operator

1. Names starting with 'A'

```
12 SELECT * FROM Employee_Details
13 WHERE Name LIKE 'A%';
```

Query result				Script output	DBMS output	Explain Plan	SQL history
  Download  Execution time: 0.008 seconds							
	EMPID	NAME	CITY				
1	1	Ananya	Delhi				
2	3	Amit	Delhi				
3	5	Arjun	Delhi				
4	1	Ananya	Delhi				
5	3	Amit	Delhi				
6	5	Arjun	Delhi				

2. Names ending with 'a'

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Employee_Details (  
2     EmpID INT,  
3     Name VARCHAR(50),  
4     City VARCHAR(50)  
5 );  
6 INSERT INTO Employee_Details VALUES  
7 (1, 'Ananya', 'Delhi'),  
8 (2, 'Rahul', 'Mumbai'),  
9 (3, 'Amit', 'Delhi'),  
10 (4, 'Neha', 'Chennai'),  
11 (5, 'Arjun', 'Delhi');  
12 SELECT * FROM Employee_Details  
13 WHERE Name LIKE '%a';
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.004 seconds

	EMPID	NAME	CITY
1	1	Ananya	Delhi
2	4	Neha	Chennai
3	1	Ananya	Delhi
4	4	Neha	Chennai
5	1	Ananya	Delhi
6	4	Neha	Chennai

3. Names containing 'hu'

```
12 SELECT * FROM Employee_Details
13 WHERE Name LIKE '%hu%';
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.122 seconds

	EMPID	NAME	CITY
1	2	Rahul	Mumbai
2	2	Rahul	Mumbai
3	2	Rahul	Mumbai
4	2	Rahul	Mumbai

4. Names with exactly 5 characters

```
12 SELECT * FROM Employee_Details
13 WHERE Name LIKE '____';
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.004 seconds

	EMPID	NAME	CITY
1	2	Rahul	Mumbai
2	5	Arjun	Delhi
3	2	Rahul	Mumbai
4	5	Arjun	Delhi
5	2	Rahul	Mumbai
6	5	Arjun	Delhi
7	2	Rahul	Mumbai

5. City starting with 'D'

```
12 SELECT * FROM Employee_Details
13 WHERE City LIKE 'D%';
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.008 seconds

	EMPID	NAME	CITY
1	1	Ananya	Delhi
2	3	Amit	Delhi
3	5	Arjun	Delhi
4	1	Ananya	Delhi
5	3	Amit	Delhi
6	5	Arjun	Delhi
7	1	Ananya	Delhi

Conclusion- The LIKE operator was successfully used to search and filter records based on specific patterns.

Experiment No – 8

Aim- To study and implement different types of SQL Constraints in table creation.

Theory

Constraints are rules applied on table columns to ensure data integrity and accuracy.

Common SQL Constraints:

- **NOT NULL** → Column cannot have NULL values
- **UNIQUE** → All values must be different
- **PRIMARY KEY** → Uniquely identifies each record (NOT NULL + UNIQUE)
- **FOREIGN KEY** → Links two tables
- **CHECK** → Restricts values based on condition
- **DEFAULT** → Sets default value

Procedure

1. Open SQL environment
2. Create tables with constraints
3. Insert data
4. Observe constraint behavior

Table Creation with Constraints

Table 1: Course

[SQL Worksheet]*      Aa  

```
1 CREATE TABLE Course (  
2     CourseID INT PRIMARY KEY,  
3     CourseName VARCHAR(50) NOT NULL  
4 );
```

Table 2: Student_Info

```
5 CREATE TABLE Student_Info (  
6     RollNo INT PRIMARY KEY,  
7     Name VARCHAR(50) NOT NULL,  
8     Email VARCHAR(50) UNIQUE,  
9     Age INT CHECK (Age >= 18),  
10    CourseID INT,  
11    City VARCHAR(50) DEFAULT 'Mumbai',  
12    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
13 );
```

Insert Data

Course Table

```
14 INSERT INTO Course VALUES  
15 (1, 'BCA'),  
16 (2, 'BBA'),  
17 (3, 'BCom');
```

Student_Info Table

```
18 INSERT INTO Student_Info (RollNo, Name, Email, Age, CourseID)  
19 VALUES (101, 'Riya', 'riya@gmail.com', 19, 1);  
20  
21 INSERT INTO Student_Info (RollNo, Name, Email, Age, CourseID)  
22 VALUES (102, 'Kabir', 'kabir@gmail.com', 20, 2);
```

Constraint Testing

✗ NOT NULL Violation

```
23 INSERT INTO Student_Info (RollNo, Name)
24 VALUES (103, 'Aman');
25 SELECT * FROM Student_Info;
```

Query result						
Script output						
DBMS output						
Execution time: 0.001 seconds						
	ROLLNO	NAME	EMAIL	AGE	COURSEID	CITY
1	103	Aman	(null)		(null)	(null) Mumbai

✗ UNIQUE Violation

```
23 INSERT INTO Student_Info (RollNo, Name, Email, Age, CourseID)
24 VALUES (104, 'Rohit', 'riya@gmail.com', 21, 1);
25 SELECT * FROM Student_Info;
```

Query result						
Script output						
DBMS output						
Execution time: 0.001 seconds						
	ROLLNO	NAME	EMAIL	AGE	COURSEID	CITY
1	103	Aman	(null)		(null)	(null) Mumbai

✗ CHECK Violation

```
23 INSERT INTO Student_Info VALUES
24 (105, 'Neha', 'neha@gmail.com', 16, 2, 'Pune');
25 SELECT * FROM Student_Info;
```

Query result						
Script output						
DBMS output						
Execution time: 0.001 seconds						
	ROLLNO	NAME	EMAIL	AGE	COURSEID	CITY
1	103	Aman	(null)		(null)	(null) Mumbai

Conclusion- Constraints are enforced only when valid values are provided; NULL values may bypass certain constraints.

Experiment No – 9

Aim- To study and implement SQL Aggregate Functions such as COUNT, SUM, AVG, MAX, MIN.

Theory

Aggregate functions are used to perform calculations on a set of values and return a single result.

- **COUNT()** → Counts number of records
- **SUM()** → Calculates total value
- **AVG()** → Calculates average
- **MAX()** → Finds maximum value
- **MIN()** → Finds minimum value

Procedure

- 1. Open SQL environment**
- 2. Create a table**
- 3. Insert sample data**
- 4. Apply aggregate functions**
- 5. Observe results**

Table Creation

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Sales (  
2     SaleID INT,  
3     ProductName VARCHAR(50),  
4     Quantity INT,  
5     Price INT  
6 );
```

Insert Data

```
7 INSERT INTO Sales VALUES  
8 (1, 'Laptop', 2, 60000),  
9 (2, 'Mouse', 5, 500),  
10 (3, 'Keyboard', 3, 1500),  
11 (4, 'Monitor', 2, 12000),  
12 (5, 'Printer', 1, 8000);
```

Aggregate Function Queries

1. COUNT()

```
13 SELECT COUNT(*) AS Total_Records FROM Sales;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.116 seconds

	TOTAL_RECORDS
1	10

2. SUM()

```
13 SELECT SUM(Price) AS Total_Price FROM Sales;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.003 seconds

	TOTAL_PRICE
1	246000

3. AVG()

```
13 SELECT AVG(Price) AS Average_Price FROM Sales;
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download 

Execution time: 0.005 seconds

	AVERAGE_PRICE
1	16400

4. MAX()

```
13 SELECT MAX(Price) AS Highest_Price FROM Sales;
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

▼

Execution time: 0.004 seconds

	HIGHEST_PRICE
1	60000

5. MIN()

```
13 SELECT MIN(Price) AS Lowest_Price FROM Sales;
```

Query resultScript outputDBMS outputExplain PlanSQL history



Download

Execution time: 0.003 seconds

	LOWEST_PRICE
1	500

6. Aggregate with Condition

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Sales (  
2     SaleID INT,  
3     ProductName VARCHAR(50),  
4     Quantity INT,  
5     Price INT  
6 );  
7 INSERT INTO Sales VALUES  
8 (1, 'Laptop', 2, 60000),  
9 (2, 'Mouse', 5, 500),  
10 (3, 'Keyboard', 3, 1500),  
11 (4, 'Monitor', 2, 12000),  
12 (5, 'Printer', 1, 8000);  
13 SELECT SUM(Quantity) AS Total_Quantity  
14 FROM Sales  
15 WHERE Price > 1000;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download 

Execution time: 0.007 seconds

	TOTAL_QUANTITY
1	56

Conclusion

Aggregate functions were successfully used to perform calculations on table data.

Experiment No – 10

Aim - To study and implement Subqueries in SQL.

Theory

A Subquery (nested query) is a query inside another SQL query. It is used to retrieve data that will be used by the main query.

Types of Subqueries:

- Single-row subquery → Returns one value
- Multi-row subquery → Returns multiple values

Procedure

1. Open SQL environment
2. Create tables
3. Insert data
4. Apply subqueries
5. Observe results

Table Creation

1 Product Table

[SQL Worksheet]*      Aa  

```
1 CREATE TABLE product (  
2     product_id NUMBER PRIMARY KEY,  
3     product_name VARCHAR2(50),  
4     price NUMBER(10,2)  
5 );
```

2 Customer Table

```
CREATE TABLE customer (  
    customer_id NUMBER PRIMARY KEY,  
    customer_name VARCHAR2(50),  
    city VARCHAR2(50)  
);
```

Insert Data

```
INSERT ALL  
    INTO product VALUES (1, 'Phone', 699.99)  
    INTO product VALUES (2, 'Laptop', 1299.99)  
    INTO product VALUES (3, 'Tablet', 499.99)  
SELECT * FROM dual;  
-----
```

```
INSERT ALL  
    INTO customer VALUES (1, 'Alice', 'New York')  
    INTO customer VALUES (2, 'Bob', 'Los Angeles')  
    INTO customer VALUES (3, 'Charlie', 'Chicago')  
SELECT * FROM dual;
```

Queries (Subqueries)

1. Select customers from New York

```
17 SELECT * FROM customer  
18 WHERE city = 'New York';
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

Execution time: 0.002 seconds

	CUSTOMER_ID	CUSTOMER_NAME	CITY
1	1	Alice	New York

2. Product with highest price (Subquery)


```

12 COMMIT;
13 SELECT product_name, price
14 FROM product
15 WHERE price = (SELECT MAX(price) FROM product);

```

Query result Script output DBMS output Explain Plan SQL history			
  Download ▼ Execution time: 0.007 seconds			
	PRODUCT_NAME		PRICE
1	Laptop		1299.99

3. Average Price of Products

```

13 SELECT AVG(price) AS average_price
14 FROM product;

```

Query result Script output DBMS output Explain Plan SQL history			
  Download ▼ Execution time: 0.005 seconds			
	AVERAGE_PRICE		
1	333.32333333333334		

4. Products Above Average Price (Subquery)

```
13 SELECT product_name, price
14 FROM product
15 WHERE price > (SELECT AVG(price) FROM product);|
```

Query result Script output DBMS output Explain Plan SQL history				
  Download Execution time: 0.007 seconds				
	PRODUCT_NAME	PRICE		
1	Laptop	1299.99		

Conclusion

Subqueries were successfully implemented to retrieve data based on results of another query.

Experiment No – 11

Aim -To study and implement various string functions in SQL.

Theory

String functions are used to manipulate and process text data in SQL.

Common String Functions:

- **UPPER()** → Converts text to uppercase
- **LOWER()** → Converts text to lowercase
- **LENGTH()** → Returns length of string
- **SUBSTR()** → Extracts part of string
- **INSTR()** → Finds position of substring
- **CONCAT()** → Joins two strings

Procedure

- 1. Open SQL environment**
- 2. Create a table**
- 3. Insert data**
- 4. Apply string functions**
- 5. Observe results**

Table Creation

[SQL Worksheet]*



Aa



```
1 CREATE TABLE Books (  
2     BookID NUMBER,  
3     Title VARCHAR2(50),  
4     Author VARCHAR2(50)  
5 );|
```

Insert Data

```
6 INSERT ALL  
7     INTO Books VALUES (1, 'Harry Potter', 'J.K. Rowling')  
8     INTO Books VALUES (2, 'The Alchemist', 'Paulo Coelho')  
9     INTO Books VALUES (3, 'Wings of Fire', 'A.P.J. Abdul Kalam')  
10 SELECT * FROM dual;  
11  
12 COMMIT;|
```

String Function Queries

1. UPPER()

```
13 SELECT UPPER(Title) FROM Books;
```

Query result

Script output

DBMS output

Explain Plan



Download ▼

Execution time: 0.007 seconds

	UPPER(TITLE)
1	HARRY POTTER
2	THE ALCHEMIST
3	WINGS OF FIRE
4	HARRY POTTER
5	THE ALCHEMIST
6	WINGS OF FIRE

2. LOWER()

```
13 SELECT LOWER(Author) FROM Books;
```

Query result

Script output

DBMS output

Explain Plan



Download ▼

Execution time: 0.005 seconds

	LOWER(AUTHOR)
1	j.k. rowling
2	paulo coelho
3	a.p.j. abdul kalam
4	j.k. rowling
5	paulo coelho
6	a.p.j. abdul kalam
7	j.k. rowling

3. LENGTH()

```
13 SELECT Title, LENGTH(Title) FROM Books;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.005 seconds

	TITLE	LENGTH(TITLE)
1	Harry Potter	12
2	The Alchemist	13
3	Wings of Fire	13
4	Harry Potter	12
5	The Alchemist	13
6	Wings of Fire	13
7	Harry Potter	12

4. SUBSTR()

👉 First 5 characters of Title

```
13 SELECT SUBSTR(Title, 1, 5) FROM Books;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.005 seconds

	SUBSTR(TITLE,1,5)
1	Harry
2	The A
3	Wings
4	Harry
5	The A
6	Wings
7	Harry

5. INSTR()

👉 Position of 'a' in Title


```
13 SELECT INSTR(Title, 'a') FROM Books;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.005 seconds

	INSTR(TITLE,'A')
1	2
2	0
3	0
4	2
5	0
6	0
7	2

6. CONCAT()

```
13 SELECT CONCAT(Title, Author) FROM Books;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download

Execution time: 0.123 seconds

	CONCAT(TITLE,AUT
1	Harry PotterJ.K. Rowl
2	The AlchemistPaulo G
3	Wings of FireA.P.J. Al
4	Harry PotterJ.K. Rowl
5	The AlchemistPaulo G
6	Wings of FireA.P.J. Al
7	Harry PotterJ.K. Rowl

7. Using || (Concatenation operator – better in Oracle)

```
13 SELECT Title || ' by ' || Author FROM Books;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download

Execution time: 0.006 seconds

	TITLE 'BY' AUTHOR
1	Harry Potter by J.K. Rowling
2	The Alchemist by Paulo Coelho
3	Wings of Fire by A.P.J. Abdul Kalam
4	Harry Potter by J.K. Rowling
5	The Alchemist by Paulo Coelho
6	Wings of Fire by A.P.J. Abdul Kalam
7	Harry Potter by J.K. Rowling

Conclusion

String functions were successfully used to manipulate and analyze text data in SQL.